Reporte técnico: Editor de texto y gestor de impresión

Oct 26, 2026 · @Aarón

Datos del proyecto

Proyecto	Integrador, Unidad 4: Pilas y Colas
Asignatura	Estructuras de Datos y Compiladores
Docente	Dra. Blanca Guadalupe Estrada Rentería
Institución	Global University
Programa	Ing. en Seguridad Tecnológica y Desarrollo de Software
Integrantes	Karina Patricia Ibarra Morales · Aarón Salvador Castañeda Ruiz
Presentación	Lunes 26 de octubre de 2026
Lenguaje	C, compilado con GCC
Repositorio	github.com/KaraIbr/Proyecto_Pilas_Colas

Planteamiento del problema

La aplicación resuelve dos problemas que aparecen juntos en el trabajo diario con documentos, y que necesitan estructuras de datos opuestas.

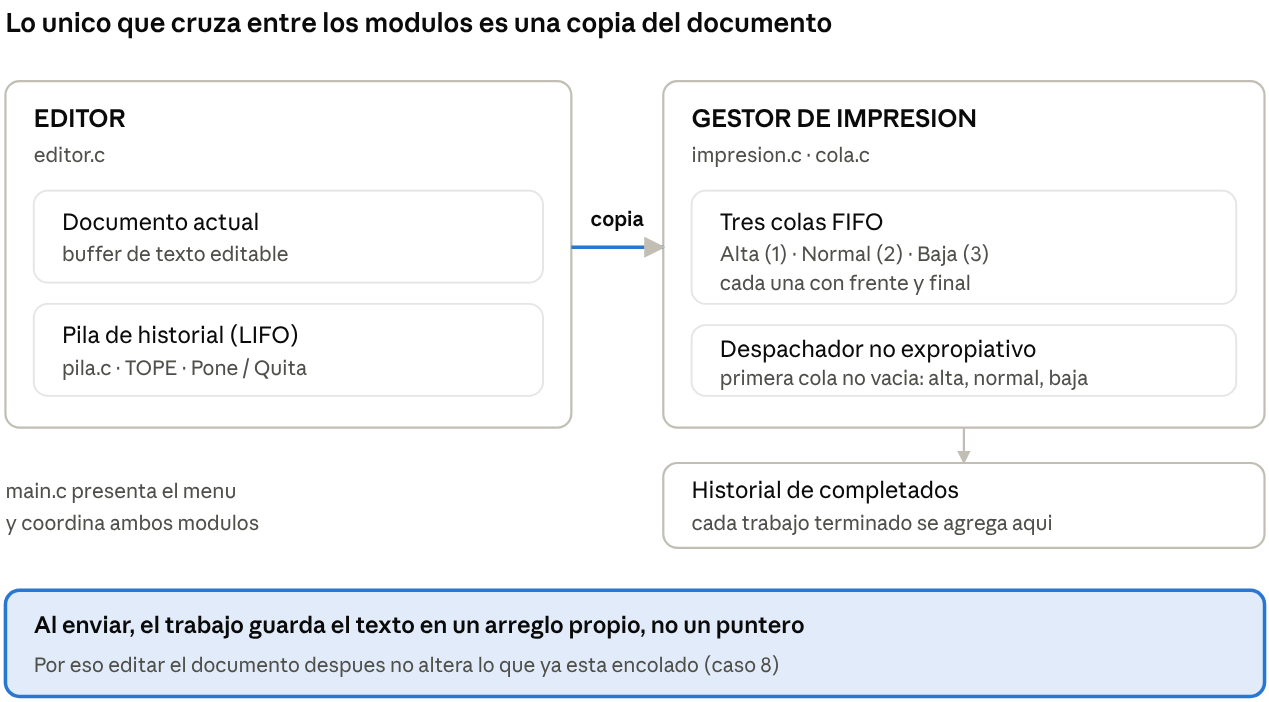
Deshacer cambios. Un editor debe poder revertir la última modificación, y luego la anterior, y así hacia atrás. El orden de recuperación es inverso al de ejecución: lo último que se hizo es lo primero que se deshace. Eso es una **pila**, con disciplina LIFO.

Administrar trabajos de impresión. Una impresora compartida recibe documentos de varios usuarios mientras aún tiene pendientes. Debe atenderlos por orden de llegada, pero respetando niveles de urgencia, y sin interrumpir lo que ya empezó a imprimir. Eso es una **cola**, con disciplina FIFO, replicada por nivel de prioridad.

La aplicación integra ambos módulos: el usuario prepara un documento en el editor, lo corrige con Deshacer y lo envía a la cola de impresión. Al enviarlo se registra una copia de su contenido en ese momento, de modo que editarlo después no altera el trabajo ya encolado.

El problema de fondo no es construir las dos estructuras por separado, sino **elegir la correcta para cada necesidad y sostener la elección**: por qué el historial no puede ser una cola y por qué la impresora no puede ser una pila.

Arquitectura general



arquitectura general · dos módulos, un punto de unión

Los dos módulos son independientes: el editor no conoce las colas y el gestor no conoce el buffer del editor. `main.c` presenta el menú y los coordina. El único tráfico entre ellos ocurre al enviar un documento a impresión, y lo que cruza es una copia de su contenido.

Diseño de la pila de historial (LIFO)

La pila se implementa con lista enlazada y **un solo punto de acceso**, el `TOPE`. Entrar y salir ocurren por el mismo extremo, y de ahí sale la disciplina LIFO.

```
   TOPE ---> [ "Red segura" ]
             [ "Red"        ]
             [ ""           ]
```

Antes de cada modificación válida del documento se apila una copia del estado previo. Al deshacer se extrae el del TOPE, que es el más reciente.

Pone (push)

El nodo nuevo apunta al antiguo tope y pasa a ser el tope:

```
nuevo->siguiente = p->tope;
p->tope = nuevo;
p->tamanio++;
```

Quita (pop)

Se devuelve una copia del texto del tope y el tope retrocede al siguiente:

```
nodo = p->tope;
/* ... copia del texto ... */
p->tope = nodo->siguiente;
p->tamanio--;
free(nodo);
```

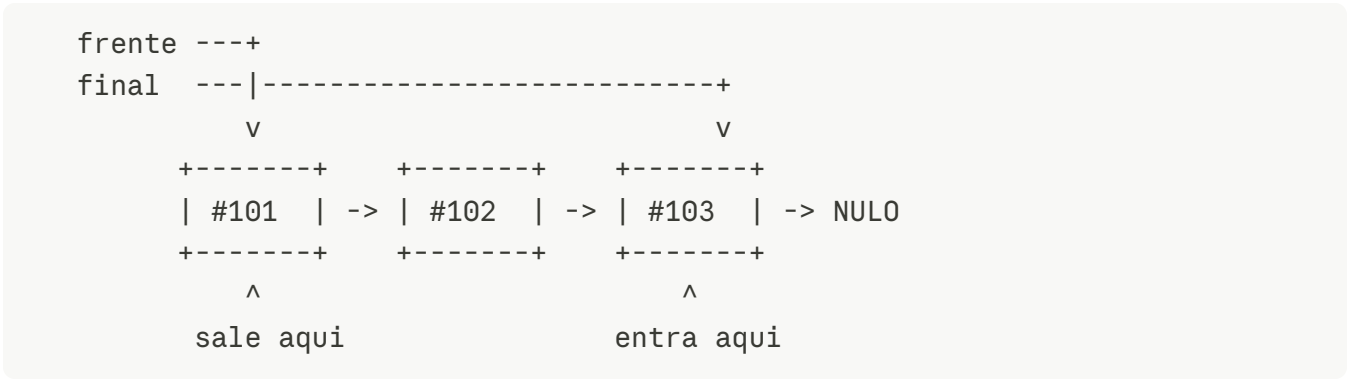
Reglas que el módulo respeta

- Solo se apila **antes de una modificación válida**. Si no hay texto que borrar, no se guarda ningún estado.
- **Consultar no apila.** Mostrar el texto o mostrar el historial recorren la estructura sin alterarla.
- **Deshacer no apila.** Extrae del TOPE y no genera un estado nuevo, para que deshacer varias veces seguidas retroceda de verdad.
- Con la pila vacía, Deshacer informa al usuario y deja el documento intacto.
- Al crear un documento nuevo se libera la pila y el historial arranca vacío.

Las tres primeras son las que distinguen un historial correcto de uno que crece sin control: si consultar o deshacer apilaran, la pila nunca se vaciaría y Deshacer dejaría de retroceder.

Diseño de la cola de impresión (FIFO)

La cola se implementa con lista enlazada y **dos puntos de acceso**: se entra por un extremo y se sale por el otro. Esa es la diferencia estructural con la pila, y la razón de que el orden de salida sea el de llegada.



Encolar y desencolar

Encolar engancha el nodo al `final`; si la cola estaba vacía, el nodo es a la vez frente y final:

```
if (c->final == NULL) {
    c->frente = nuevo;
    c->final  = nuevo;
} else {
    c->final->siguiente = nuevo;
    c->final            = nuevo;
}
```

Desencolar retira el del `frente` y, si era el último, suelta también el `final`:

```
retirado  = c->frente;
*salida   = retirado->trabajo;
c->frente = retirado->siguiente;
if (c->frente == NULL) {
    c->final = NULL;
}
```

Soltar el `final` es imprescindible: si se omite, queda apuntando a memoria liberada y el siguiente encolado escribe sobre ella.

Ninguna de las dos recorre la lista, así que ambas son de tiempo constante.

Las tres prioridades

El gestor mantiene **tres colas FIFO independientes**, una por nivel. El despachador atiende la primera cola no vacía en el orden alta, normal, baja.

```
ALTA   (1):  frente -> [ ] -> [ ] -> NULO
NORMAL (2):  frente -> [ ] -> NULO
BAJA   (3):  frente -> NULO
```

Se eligieron tres colas separadas en lugar de una sola lista ordenada por tres razones:

1. **La prioridad vive en la estructura, no en el dato.** Un trabajo está en la cola Alta; no necesita compararse con nadie para demostrarlo.
2. **Encolar es de tiempo constante.** Entra al final de su nivel, sin buscar posición de inserción. Una lista ordenada obliga a recorrerla en cada inserción.
3. **El orden de llegada es inalterable dentro de un nivel.** Solo se inserta por el final y solo se sale por el frente, así que ningún criterio de comparación mal escrito puede reordenar trabajos de la misma prioridad.

Datos de cada trabajo

Campo	Contenido
`id`	Clave única del trabajo
`nombre`	Nombre del documento
`paginas`	Cantidad mayor que cero, capturada por el usuario
`prioridad`	Alta (1), normal (2) o baja (3)
`orden_llegada`	Consecutivo asignado al registrarlo
`estado`	En espera, procesando o completado

Al registrarse, el trabajo queda **En espera**. Al despacharlo pasa a **Procesando**, y al finalizar queda **Completado** y se agrega al historial de impresión.

El trabajo **no guarda el contenido del documento**. Ese contenido vive en el archivo `trabajo_NN.txt`, donde `NN` es el folio, escrito en el momento del envío y leído al imprimirlo.

Política no expropiativa

Una impresión que ya comenzó no se interrumpe. El despachador comprueba la bandera de ocupada **antes** de tocar las colas:

```
if (g->ocupada) {
    return DESPACHO_OCUPADO;
}
```

Ese orden es toda la regla. Si la comprobación estuviera después del desencolado, el trabajo ya habría salido de su cola y habría que devolverlo, que es justo lo que no puede hacerse con una hoja a medio imprimir.

Integración de los módulos

Los dos módulos funcionan como una sola aplicación de consola. El menú agrupa las operaciones del editor (1 a 7) y las del gestor de impresión (8 a 12).

El punto de unión es el envío a impresión. Al enviar, la aplicación escribe el contenido del documento en un archivo propio del trabajo, y el gestor encola solo sus datos de control.

El nombre del archivo se deriva del folio que el gestor asigna al trabajo:

```
sprintf(archivo, "trabajo_%02d.txt", registrado.id);
guardarDocumento(e, archivo);
```

El archivo se escribe una sola vez, en el momento del envío, y nadie lo vuelve a tocar. Al imprimir, el contenido se lee de ahí. Si la cola guardara un puntero al texto vivo del editor, editar el documento después cambiaría lo que la impresora va a imprimir.

Dicho de otro modo: **los dos módulos no comparten memoria ni estructura**. Lo único que cruza entre ellos es un archivo escrito en el instante del envío. Por eso el caso de prueba 8 se cumple por diseño: el archivo se escribe una sola vez y nadie lo vuelve a tocar, así que no hay ningún punto del código donde haya que acordarse de preservar el texto.

El número de páginas se captura manualmente, sin calcularlo a partir del texto, y se valida junto con la prioridad antes de encolar.

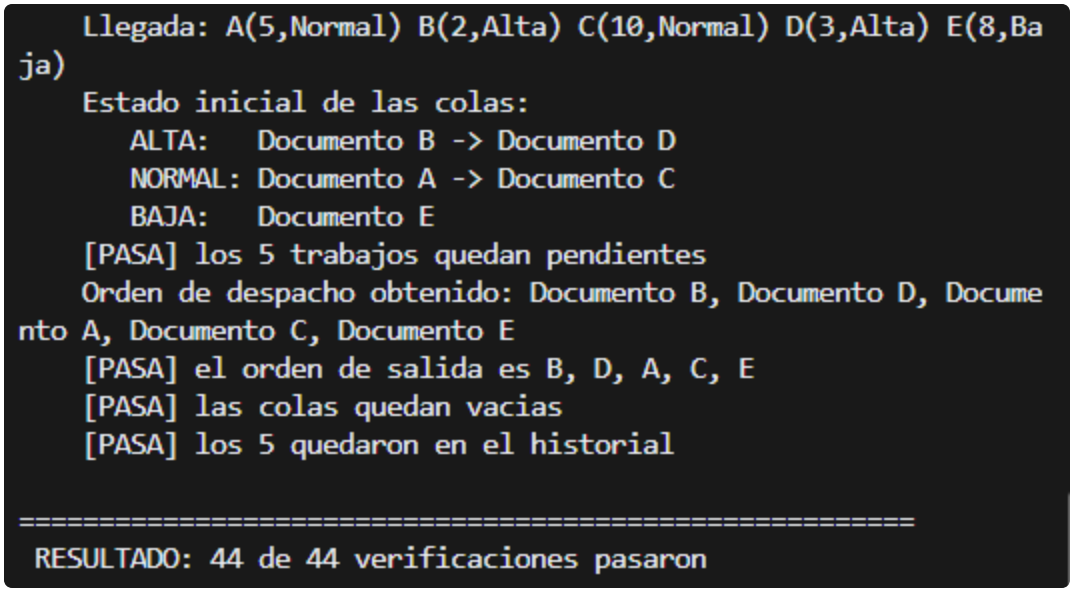
Evidencias de pruebas

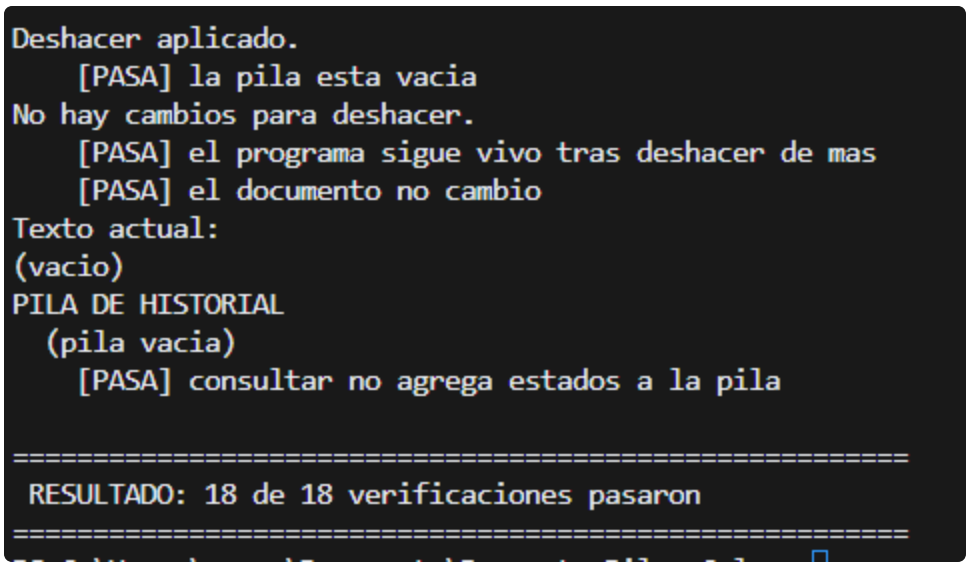
Las pruebas se ejecutaron por dos vías: capturas de la aplicación corriendo, guardadas en `capturas/`, y dos baterías automatizadas que devuelven código de salida distinto de cero si alguna verificación falla.

Baterías automatizadas

Batería	Comando	Resultado
Gestor de impresión	`make correr-pruebas`	42 de 42
Editor y pila	`make pruebas-editor`	22 de 22

64 verificaciones, todas en verde. Las salidas completas de consola están en `docs/salida_pruebas_impresion.txt` y `docs/salida_pruebas_editor.txt`.





Casos obligatorios

#	Caso	Comportamiento observado
1	Varios trabajos con la misma prioridad	Salen en orden de llegada: Doc-1, Doc-2, Doc-3
2	Trabajos con prioridades diferentes	Alta-1 se atiende primero aunque llegó al final
3	Llega un trabajo urgente después	Se coloca al final de su nivel y no interrumpe el trabajo en curso
4	Se intenta procesar sin trabajos	Informa que las colas están vacías, sin error
5	Se realizan varias modificaciones	El historial conserva los estados anteriores
6	Se ejecuta Deshacer varias veces	Los estados se recuperan en orden LIFO
7	Se ejecuta Deshacer con pila vacía	Avisa y no modifica el documento
8	Se envía un documento editado	El trabajo encolado conserva el texto original

Caso 1 · Varios trabajos con la misma prioridad

Tres documentos encolados en Normal salen en el orden en que llegaron.

```
Seleccione una opcion: 12

═══════════════════════════════════════════════════════════

--- HISTORIAL DE TRABAJOS COMPLETADOS ---
  1. #1  doc-1                3 pag.  prioridad NORMAL  Comple
tado
  2. #2  doc-2                1 pag.  prioridad NORMAL  Comple
tado
  3. #3  doc-3                7 pag.  prioridad NORMAL  Comple
tado
  Total completados: 3
```

Caso 2 · Trabajos con prioridades diferentes

Baja-1, Normal-1 y Alta-1 se registran en ese orden; cada uno entra a la cola de su nivel.

```
--- ESTADO DEL GESTOR DE IMPRESION ---
  EN CURSO: impresora libre
  [ALTA  ] 1 pendiente(s)
      1. #3  Alta-1               2 pag.  llegada 3  En espera
  <- FRENTE
  [NORMAL] 1 pendiente(s)
      1. #2  Normal-1             2 pag.  llegada 2  En espera
  <- FRENTE
  [BAJA  ] 1 pendiente(s)
      1. #1  Baja-1               2 pag.  llegada 1  En espera
  <- FRENTE
  SIGUIENTE POR ATENDER: #3 Alta-1 (prioridad ALTA)
  Pendientes: 3    Completados: 0
-------------------------------------
```

Al despacharlos, Alta-1 sale primero aunque fue el último en llegar.

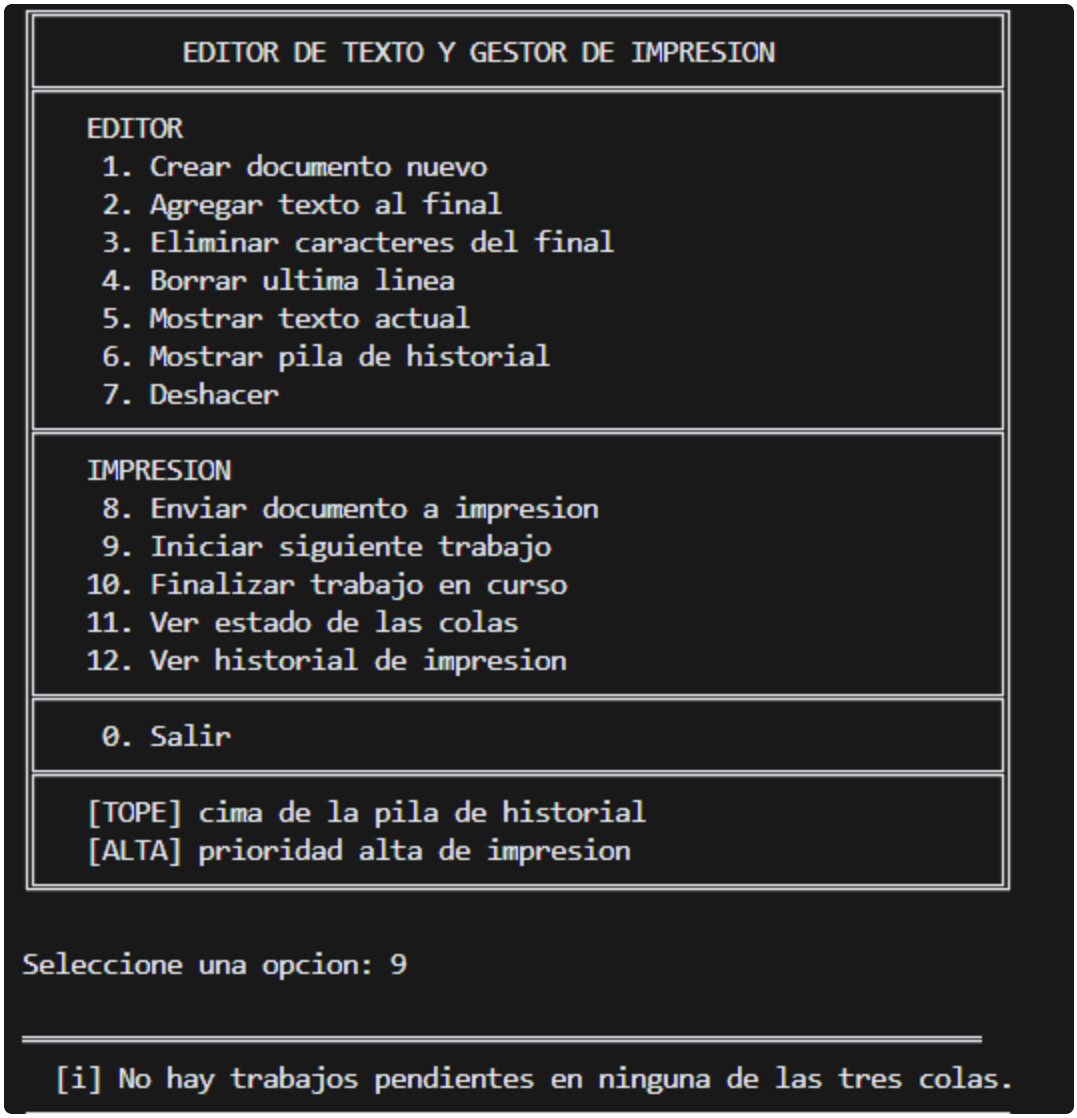
```
Seleccione una opcion: 12

═══════════════════════════════════════════════════════════

--- HISTORIAL DE TRABAJOS COMPLETADOS ---
  1. #3  Alta-1               2 pag.  prioridad ALTA    Comple
tado
  2. #2  Normal-1             2 pag.  prioridad NORMAL  Comple
tado
  3. #1  Baja-1               2 pag.  prioridad BAJA    Comple
tado
  Total completados: 3
```

Caso 4 · Procesar sin trabajos pendientes

Con las tres colas vacías, el programa informa y continúa sin error.



Caso 5 · Varias modificaciones consecutivas

Cada adición guarda el estado previo en la pila antes de modificar el documento.



Caso 6 · Deshacer varias veces seguidas

Cada Deshacer recupera el estado más reciente de la pila, en orden LIFO.

```
Seleccione una opcion: 7

═══════════════════════════════════════════════════

Deshacer aplicado.

┌──────────────────────────────────────────────┐
│        EDITOR DE TEXTO Y GESTOR DE IMPRESION │
├──────────────────────────────────────────────┤
│    3. Eliminar caracteres del final          │
│    4. Borrar ultima linea                    │
│    5. Mostrar texto actual                   │
│    6. Mostrar pila de historial              │
│    7. Deshacer                               │
├──────────────────────────────────────────────┤
│   IMPRESION                                  │
│    8. Enviar documento a impresion           │
│    9. Iniciar siguiente trabajo              │
│   10. Finalizar trabajo en curso             │
│   11. Ver estado de las colas                │
│   12. Ver historial de impresion             │
├──────────────────────────────────────────────┤
│    0. Salir                                  │
├──────────────────────────────────────────────┤
│   [TOPE] cima de la pila de historial        │
│   [ALTA] prioridad alta de impresion         │
└──────────────────────────────────────────────┘

Seleccione una opcion: 5

═══════════════════════════════════════════════════

Texto actual:
12

┌──────────────────────────────────────────────┐
│        EDITOR DE TEXTO Y GESTOR DE IMPRESION │
├──────────────────────────────────────────────┤
│   EDITOR                                     │
│    1. Crear documento nuevo                  │
│    2. Agregar texto al final                 │
│    3. Eliminar caracteres del final          │
│    4. Borrar ultima linea                    │
│    5. Mostrar texto actual                   │
│    6. Mostrar pila de historial              │
│    7. Deshacer                               │
├──────────────────────────────────────────────┤
│   IMPRESION                                  │
│    8. Enviar documento a impresion           │
│    9. Iniciar siguiente trabajo              │
│   10. Finalizar trabajo en curso             │
│   11. Ver estado de las colas                │
```

```
   11. Ver estado de las colas
   12. Ver historial de impresion

    0. Salir

   [TOPE] cima de la pila de historial
   [ALTA] prioridad alta de impresion
```

Caso 7 · Deshacer con la pila vacía

Sin estados guardados, el programa avisa y deja el documento intacto.

```
Seleccione una opcion: 7

==========

No hay cambios para deshacer.

        EDITOR DE TEXTO Y GESTOR DE IMPRESION

   EDITOR
    1. Crear documento nuevo
    2. Agregar texto al final
    3. Eliminar caracteres del final
    4. Borrar ultima linea
    5. Mostrar texto actual
    6. Mostrar pila de historial
    7. Deshacer

   IMPRESION
    8. Enviar documento a impresion
    9. Iniciar siguiente trabajo
   10. Finalizar trabajo en curso
   11. Ver estado de las colas
   12. Ver historial de impresion

    0. Salir

   [TOPE] cima de la pila de historial
   [ALTA] prioridad alta de impresion

Seleccione una opcion: 6

==========

PILA DE HISTORIAL
  (pila vacia)
```

Caso 8 · Documento editado después de enviarlo

Tras enviar el documento a impresión se edita su contenido.

```
Seleccione una opcion: 5

═══════════════════════════════════════════════════════

Texto actual:
Contrato version 1EDITADO DESPUES
```

El archivo escrito al enviarlo conserva el texto original.

```
Seleccione una opcion: 11

═══════════════════════════════════════════════════════

--- ESTADO DEL GESTOR DE IMPRESION ---
  EN CURSO: impresora libre
  [ALTA   ] 0 pendiente(s) -- vacia
  [NORMAL] 1 pendiente(s)
      1. #1  Contrato              2 pag.  llegada 1  En espera
  <- FRENTE
  [BAJA   ] 0 pendiente(s) -- vacia
  SIGUIENTE POR ATENDER: #1 Contrato  (prioridad NORMAL)
  Pendientes: 1    Completados: 0
```

El archivo del trabajo, paso a paso

La secuencia completa de la persistencia en disco, desde el envío hasta la impresión.

Al enviar el documento se escribe su archivo y el programa informa cuántos caracteres guardó.

```
Seleccione una opcion: 8

═══════════════════════════════════════════════════════

  Nombre del documento: Contrato
  Numero de paginas (> 0): 2
  Prioridad (1=Alta, 2=Normal, 3=Baja): 2
  [OK] Trabajo #1 encolado en prioridad NORMAL [En espera].
       Documento guardado en trabajo_01.txt (20 caracteres).
```

El archivo queda en el directorio desde el que se ejecutó el programa, con los mismos 20 bytes.

```
  Saliendo. Memoria liberada.
PS C:\Users\aaron\Documents\Proyecto_Pilas_Colas> dir trabajo_*.
txt


    Directorio: C:\Users\aaron\Documents\Proyecto_Pilas_Colas


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
-a----     26/10/2026  07:19 p. m.             20 trabajo_01.tx
                                                  t


PS C:\Users\aaron\Documents\Proyecto_Pilas_Colas>
```

Después se edita el documento, que pasa a ser más largo que lo guardado.

```
 Seleccione una opcion: 5

 ═══════════════════════════════════════════════════════════════
 Texto actual:
 Contrato version unoY EDICIÓN POSTERIOR
```

Al imprimir, el contenido se recupera del archivo: la versión original, no la editada.

```
Seleccione una opcion: 9

═══════════════════════════════════════════════════════════════
  [OK] Imprimiendo #1 Contrato  (2 pag., prioridad NORMAL) [Proc
esando].
       Contenido recuperado de trabajo_01.txt:
         Contrato version uno
       Sigue en curso hasta que se finalice.
```

Al finalizar, el trabajo pasa a Completado y entra al historial.

```
Seleccione una opcion: 10

═══════════════════════════════════════════════════════════════
  [OK] Trabajo #1 Contrato  -> Completado. Agregado al historial
.
```

Image — Historial de impresión con el trabajo Contrato completado

Los números coinciden entre las tres capturas clave: 20 caracteres al guardar, 20 bytes en el disco y el mismo texto al imprimir.

Caso 3 en detalle

Es el que sostiene la política no expropiativa. Con un trabajo de prioridad baja en curso y uno de prioridad alta recién encolado, el despachador responde:

```
[i] La impresora esta ocupada con #1 Informe-Baja.
    El trabajo en curso no se interrumpe aunque haya
    pendientes de mayor prioridad.
```

El estado de las colas confirma que el urgente quedó en espera mientras el trabajo en curso sigue en Procesando. Al finalizarlo, el urgente entra de inmediato.

Image — Mensaje de que la impresora esta ocupada y el trabajo en curso no se interrumpe

Estado antes de que llegue el trabajo urgente: Informe-Baja en curso, marcado como Procesando.

Image — Estado con Informe-Baja en curso y las tres colas vacias

Con el urgente ya encolado, el trabajo en curso sigue siendo el mismo y el urgente espera en la cola Alta.

Image — Estado con Informe-Baja en curso y Urgente-Alta en espera

Al finalizar el trabajo en curso, el urgente entra de inmediato.

Image — Urgente-Alta pasa a Procesando

Caso del simulador

Con los datos del enunciado — A (5, normal), B (2, alta), C (10, normal), D (3, alta), E (8, baja), registrados en ese orden — las colas quedan así antes de atender nada:

```
ALTA:    Documento B -> Documento D
NORMAL:  Documento A -> Documento C
BAJA:    Documento E
```

Image — Estado inicial con B y D en Alta, A y C en Normal, E en Baja

El historial resultante tras despacharlos todos:

Orden de salida	Trabajo	Páginas	Prioridad
1	Documento B	2	Alta
2	Documento D	3	Alta
3	Documento A	5	Normal
4	Documento C	10	Normal
5	Documento E	8	Baja

Image — Historial con Documento B, D, A, C y E completados en ese orden

Orden obtenido: B, D, A, C, E, que coincide con el esperado. Las dos justificaciones que explica: B sale antes que A porque la prioridad manda entre niveles, y B sale antes que D porque dentro del mismo nivel manda el orden de llegada.

Conclusiones

La estructura correcta vuelve triviales los casos difíciles. El caso de prueba 8 — que un documento editado después de enviarlo no altere el trabajo encolado — no requirió ninguna lógica especial: basta con que el trabajo guarde el texto en un arreglo propio en lugar de un puntero. Si se hubiera elegido compartir el puntero, habría hecho falta recordar copiar el texto en cada punto de envío, y bastaría olvidarlo una vez para romperlo.

Separar por estructura es más robusto que ordenar por criterio. Tres colas independientes garantizan el orden de llegada dentro de cada nivel por construcción. Una lista ordenada habría dejado ese orden a merced de un criterio de comparación, que es

código que puede estar mal sin que nadie lo note.

El orden de las comprobaciones es parte del diseño. La política no expropiativa es una sola línea, pero colocada antes del desencolado. Movida después, el trabajo ya habría salido de su cola y habría que devolverlo.

Las operaciones de consulta deben ser inocuas. En la pila, que mostrar el texto o el historial no apilen estados nuevos es lo que permite que Deshacer retroceda de verdad. Una consulta que modifica la estructura convierte el historial en algo que nunca se vacía.

Las pruebas automatizadas valieron más que las manuales. Las 62 verificaciones se ejecutan en un segundo y detectaron regresiones durante la integración que una prueba manual habría pasado por alto.

Contribución de cada integrante

Karina Patricia Ibarra Morales

Módulo	Archivos
Pila de historial (LIFO)	`include/pila.h`, `src/pila.c`
Editor de documentos	`include/editor.h`, `src/editor.c`
Interfaz de consola	`include/ui.h`, `src/ui.c`
Documentación	`docs/pseudocodigo.md`, `docs/reporte-tecnico.md`

Implementó la pila con manejo explícito del TOPE y las operaciones Pone y Quita, el editor con su buffer de texto y el mecanismo de Deshacer, y la interfaz de terminal con bordes y los indicadores `[TOPE]` y `[ALTA]`.

Aarón Salvador Castañeda Ruiz

Módulo	Archivos
Cola FIFO	`include/cola.h`, `src/cola.c`
Gestor de impresión	`include/impresion.h`, `src/impresion.c`
Aplicación principal	`src/main.c`

Módulo	Archivos
Pruebas	`src/pruebas_impresion.c`, `src/pruebas_editor.c`
Compilación	`Makefile`
Documentación	`docs/pseudocodigo-cola.md`, `docs/evidencias_gestor_impresion.md`

Implementó la cola con punteros explícitos a frente y final, las tres colas de prioridad con su despachador y la política no expropiativa, el estado de los trabajos y el historial de completados, la integración de ambos módulos en el menú y las dos baterías de pruebas.

Trabajo conjunto

La integración final requirió ajustes cruzados: se agregaron al editor las operaciones de eliminar una cantidad indicada de caracteres y mostrar el contenido de la pila, y se separó el despacho en iniciar y finalizar para poder observar la política no expropiativa desde la interfaz.

El repositorio sigue una política de ramas por funcionalidad con integración mediante pull requests, de modo que el historial de commits refleja quién escribió cada parte.